

Relatório de Inteligência Artificial - CSI701

Turma 11

Alunos:

Nicholas Arthur Guimarães Andrade nº 23.2.8018

Vívian Evelyn Duarte nº 23.2.8017

1. Introdução

Este relatório apresenta o desenvolvimento de um agente inteligente capaz de atuar em labirintos discretos representados por matrizes. O mesmo domínio foi utilizado para estudar três formas diferentes de resolução de problemas em Inteligência Artificial: busca clássica, busca local e busca online. O objetivo do trabalho é modelar formalmente o agente, implementar algoritmos de busca, comparar estratégias, coletar métricas experimentais, gerar visualizações e analisar criticamente os resultados obtidos.

O agente foi estudado em três situações principais:

1. Labirinto conhecido: o agente conhece todo o mapa e deve encontrar um caminho de A até B;
2. Labirinto com pontos de coleta: o agente deve visitar todos os pontos C_i antes de chegar a B;
3. Labirinto desconhecido: o agente não conhece o mapa completo e deve explorá-lo enquanto se movimenta.

O trabalho foi desenvolvido em Python, com leitura de mapas em formato ASCII, execução dos algoritmos, geração de arquivos .csv, visualizações textuais e análise experimental.

2. Contexto do Problema

O ambiente do trabalho é um labirinto representado por uma grade bidimensional. Cada posição da matriz corresponde a uma célula do ambiente. A legenda utilizada é:

Símbolo	Significado
A	posição inicial do agente
B	objetivo final
C	ponto de coleta obrigatório
#	parede ou obstáculo
espaço em branco	célula livre
?	célula desconhecida no modo online

O agente pode se mover apenas em quatro direções ortogonais: cima, baixo, esquerda e direita. Não são permitidos movimentos diagonais, saltos sobre paredes ou deslocamentos para células não adjacentes.

3. Modelagem PEAS do Agente

A modelagem PEAS define formalmente o agente por meio de quatro componentes:

PEAS = (Performance, Environment, Actuators, Sensors)

O agente atua em um labirinto discreto e deve tomar decisões para alcançar objetivos. Esses objetivos variam conforme a etapa: alcançar B, visitar pontos C_i antes de chegar a B ou explorar um mapa inicialmente desconhecido.

3.1 Performance

A medida de desempenho define o que será considerado um bom comportamento do agente. Como o agente deve resolver labirintos, seu desempenho deve ser avaliado pelo sucesso em alcançar o objetivo, pela qualidade do caminho e pelo esforço computacional necessário.

O comportamento esperado do agente é:

- alcançar o objetivo final B;
- evitar movimentos inválidos;
- minimizar o custo total do caminho;
- reduzir o número de passos;
- reduzir o número de nós expandidos;
- reduzir o tempo de execução;
- reduzir o tamanho máximo da fronteira;
- evitar revisitas desnecessárias;
- no modo com coleta, visitar todos os pontos C_i antes de chegar a B;
- no modo online, reduzir o custo adicional causado pelo desconhecimento inicial do mapa.

Uma função geral de desempenho pode ser representada por:

$J = R_{\text{sucesso}} - \alpha \text{ custo_caminho} - \beta \text{ nós_expandidos} - \gamma \text{ tempo_execução} - \delta \text{ movimentos_inválidos} - \epsilon \text{ células_revisitadas} - \zeta \text{ replanejamentos}$

Nessa função, (R_{sucesso}) representa a recompensa por atingir corretamente o objetivo. Os demais termos representam penalizações associadas ao custo do caminho, esforço computacional, tempo, erros de movimento, revisitas e replanejamentos. Na busca clássica, a performance é avaliada por sucesso, custo, passos, nós explorados, nós expandidos, tempo de execução e tamanho máximo da fronteira. Na busca local, a performance é avaliada por melhor custo encontrado, pior custo, custo médio, tempo médio, número médio de iterações, taxa de sucesso e curva de convergência. Na busca online, a performance é avaliada por sucesso, custo real percorrido, células reveladas, células revisitadas, número de replanejamentos e razão online/offline:

razão online/offline = custo percorrido pelo agente online / custo ótimo no mapa completo

Quanto mais próxima essa razão estiver de 1, melhor será o desempenho do agente online.

3.2 Environment

O ambiente é o labirinto em que o agente atua. Ele é representado por uma matriz bidimensional formada por células livres, paredes, posição inicial, objetivo final, pontos de coleta e, no modo online, células desconhecidas. Na busca clássica, o ambiente é totalmente observável, pois o agente conhece todo o mapa desde o início. Na busca local, o ambiente também é totalmente observável, mas o objetivo do agente é decidir a ordem de visita dos pontos de coleta. Na busca online, o ambiente é parcialmente observável. O agente não conhece inicialmente o mapa completo e precisa atualizar seu modelo interno conforme percebe novas células. As propriedades do ambiente são:

Propriedade	Classificação no trabalho
Observabilidade	totalmente observável na busca clássica/local; parcialmente observável na busca online
Determinismo	determinístico
Episódico ou sequencial	sequencial
Estático ou dinâmico	o labirinto real é estático, mas o conhecimento do agente muda no modo online
Discreto ou contínuo	discreto
Conhecido ou desconhecido	conhecido na busca clássica/local; inicialmente desconhecido na busca online

O ambiente é sequencial porque cada ação altera a posição do agente e influencia as próximas decisões. É determinístico porque uma ação válida sempre leva ao

mesmo estado resultante. É discreto porque os estados e ações são finitos e bem definidos.

3.3 Actuators

Os atuadores são os comandos de movimentação que permitem ao agente mudar sua posição dentro da matriz do labirinto. O conjunto de ações do agente é: $A = \{\text{cima, baixo, esquerda, direita}\}$. Considerando a posição atual do agente como $((x,y))$, cada ação corresponde a:

cima: $(x,y) \rightarrow (x-1,y)$

baixo: $(x,y) \rightarrow (x+1,y)$

esquerda: $(x,y) \rightarrow (x,y-1)$

direita: $(x,y) \rightarrow (x,y+1)$

Cada ação só é efetivada se a célula de destino estiver dentro dos limites do labirinto e não for uma parede. Caso contrário, o movimento é inválido e o agente permanece na posição atual. Na busca clássica, os atuadores executam a sequência de ações encontrada pelo algoritmo de busca. Na busca local, os atuadores continuam sendo os mesmos, mas são usados para percorrer os caminhos entre A, os pontos C_i e B. Na busca online, os atuadores são executados passo a passo, pois o agente percebe o ambiente, atualiza seu mapa interno, planeja e executa apenas a próxima ação.

3.4 Sensors

Os sensores representam os mecanismos pelos quais o agente obtém informações do labirinto. As percepções podem incluir:

- posição atual do agente;
- conteúdo das células do labirinto;
- identificação de paredes;
- identificação de células livres;
- localização de A;
- localização de B;
- localização dos pontos C_i ;
- identificação de células desconhecidas ?;
- vizinhança local no modo online;
- histórico de células já visitadas.

Na busca clássica, os sensores fornecem acesso completo ao mapa. Na busca local, os sensores também fornecem o mapa completo e a localização dos pontos de coleta. No modo online, os sensores são limitados. Considerando raio de percepção ($r=1$), se o agente está na posição $((x,y))$, ele percebe as células ortogonais: $(x-1,y)$, $(x+1,y)$, $(x,y-1)$, $(x,y+1)$. A cada passo, o agente atualiza seu mapa interno, substituindo células desconhecidas por informações reais, como célula livre, parede, objetivo ou ponto de coleta.

3.5 Classificação do Agente

O agente desenvolvido neste trabalho é classificado como um agente baseado em objetivos com modelo interno. Ele é baseado em objetivos porque suas ações são escolhidas em função de metas: alcançar B, visitar todos os pontos C_i quando necessário e encontrar o objetivo mesmo sem conhecer todo o mapa no modo online. Ele também é baseado em modelo porque mantém uma representação interna do ambiente.

Na busca clássica e local, esse modelo corresponde ao mapa completo. Na busca online, o modelo interno começa incompleto e é atualizado progressivamente conforme o agente percebe novas células. Ele usa planejamento, busca, histórico perceptual e atualização de estado interno. O agente não é considerado aprendiz, pois não altera sua política de decisão com base em aprendizado de experiências anteriores. Ele apenas atualiza o conhecimento do mapa durante a execução.

4. Busca Clássica no Labirinto Conhecido

Nesta etapa, o agente conhece todo o labirinto desde o início. O problema consiste em encontrar um caminho válido da posição inicial A até o objetivo final B.

A $\rightarrow$ B

O agente conhece o ambiente, possui estado inicial definido, objetivo explícito, ações possíveis e pode planejar uma sequência completa de ações antes da execução.

4.1 Formulação Formal

O problema foi formulado como: $\{S, A, T, s_0, G, c\}$

onde:

- (S) é o conjunto de estados possíveis;
- (A) é o conjunto de ações disponíveis;
- (T) é a função de transição;
- (s_0) é o estado inicial;
- (G) é o teste de objetivo;
- (c) é a função de custo.

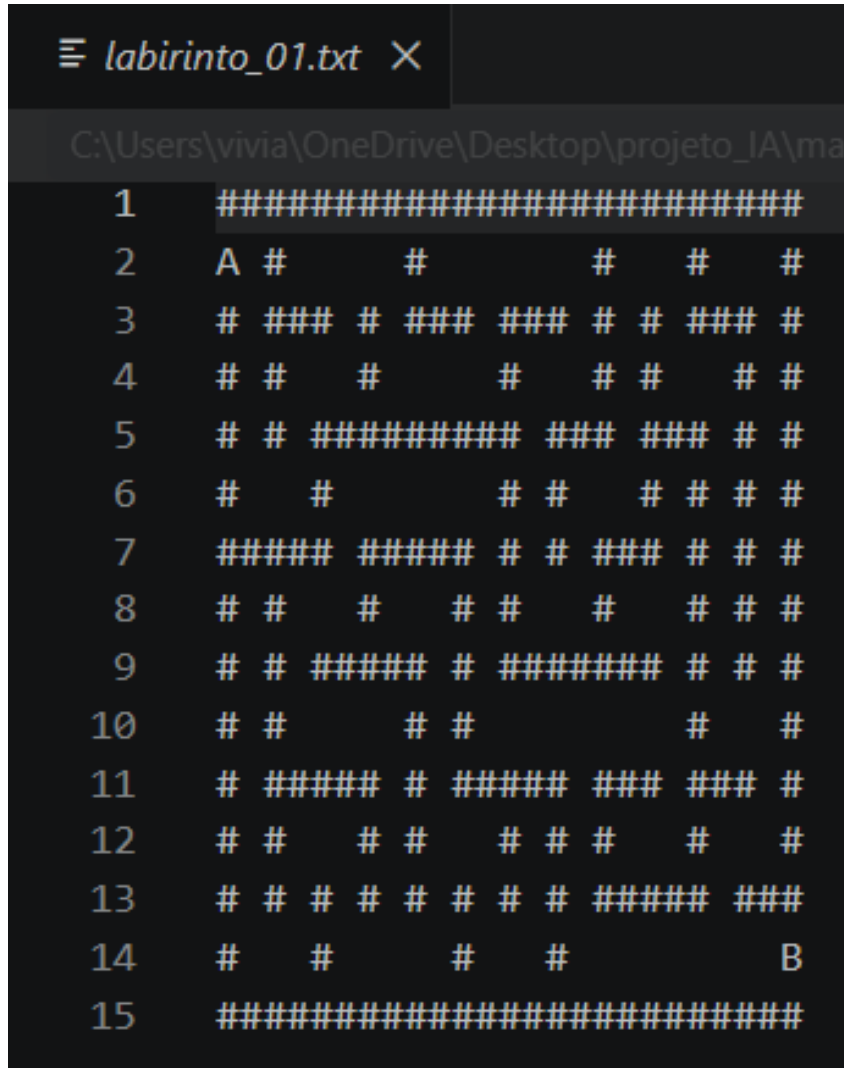
Cada estado representa uma posição do agente na matriz: $s = (x,y)$

O conjunto (S) é formado por todas as células livres do labirinto, incluindo A e B. As paredes # não pertencem ao conjunto de estados válidos. O conjunto de ações é: $A = \{\text{cima, baixo, esquerda, direita}\}$. A função de transição $T(s,a)$ retorna o novo estado após uma ação válida. O estado inicial é: $s_0 = \text{posição}(A)$. O teste de objetivo é: $G(s) = \{\text{verdadeiro se } s = \text{posição}(B) \text{ e falso caso contrário}\}$. A função de custo atribui custo unitário a cada movimento válido: $c(s,a,s') = 1$. Portanto, o custo total do caminho equivale ao número de passos executados.

4.2 Mapas Utilizados

Foram utilizados três labirintos conhecidos gerados em formato ASCII com a ferramenta ASCII Maze Generator. Os mapas foram configurados no tema clássico, usando # para paredes e espaços em branco para caminhos livres.

Imagem do labirinto 1 que está em mapas/labirinto_01.txt



```
labirinto_01.txt X
C:\Users\vivia\OneDrive\Desktop\projeto_IA\ma
1 #####
2 A #   #   #   #   #
3 # ### # ### ### # # ### #
4 # #  #   #   # #   # #
5 # # ##### ### ### # #
6 #  #   #   # #   # # #
7 ##### ##### # # ### # # #
8 # #  #  # #   #   # # #
9 # # ##### # ##### # # #
10 # #   # #   #   #   #
11 # ##### # ##### ### ### #
12 # #  # #   # # #   #   #
13 # # # # # # # # ##### ###
14 #  #   #  #   B
15 #####
```

Imagem do labirinto 2 que está em mapas/labirinto_02.txt

```
labirinto_02.txt X
mapas > labirinto_02.txt
1 #####
2 A # # # #
3 # ##### # # ##### # # #
4 # # # # # # # #
5 # # # ##### # # ##### # # #
6 # # # # # # # # # #
7 # # ##### ##### # # # # # #
8 # # # # # # # # # #
9 # # # # # # # # # #
10 # # # # # # # #
11 ##### # # # ##### # #
12 # # # # # # # #
13 ##### # # # # # # # #
14 # # # # # # # # # #
15 # ##### # # # # # # #
16 # # # # # # # # #
17 # # # # # # # # # #
18 # # # # # # # # # #
19 # ##### # # # # # # #
20 # # # # # # # # #
21 ##### # # # # # # # #
22 # # # # # # # # # #
23 # # # # ##### ##### # # #
24 # # # # # # # # #
25 # ##### ##### # # #
26 # # B
27 #####
```


BFS, DFS e UCS são buscas não informadas. Busca Gulosa e A* são buscas informadas, pois utilizam uma heurística. A heurística utilizada foi a distância de Manhattan: $h(n) = |x_n - x_B| + |y_n - y_B|$. Essa heurística é adequada porque o agente só se move em direções ortogonais e cada movimento possui custo unitário.

4.4 Métricas Coletadas

Para cada algoritmo, foram coletadas as seguintes métricas:

- sucesso;
- custo;
- passos;
- nós explorados;
- nós expandidos;
- tempo de execução;
- tamanho máximo da fronteira.

Os “nós explorados” representa a quantidade de nós retirados da fronteira para análise, enquanto “nós expandidos” representa a quantidade de nós cujos sucessores foram gerados.

4.5 Resultados Experimentais

Mapa	Algoritmo	Sucesso	Custo	Passos	Explorados	Expandidos	Tempo (s)	Fronteira
Labirinto 1	BFS	TRUE	68	68	85	84	0.00019610	3
Labirinto 1	DFS	TRUE	68	68	169	168	0.00025730	4
Labirinto 1	UCS	TRUE	68	68	85	84	0.00014920	3
Labirinto 1	Busca Gulosa	TRUE	68	68	69	68	0.00012340	5
Labirinto 1	A*	TRUE	68	68	75	74	0.00012550	3
Labirinto 2	BFS	TRUE	314	314	494	493	0.00085260	4
Labirinto 2	DFS	TRUE	314	314	477	476	0.00125200	7
Labirinto 2	UCS	TRUE	314	314	494	493	0.00085810	4
Labirinto 2	Busca Gulosa	TRUE	314	314	357	356	0.00065800	9
Labirinto 2	A*	TRUE	314	314	467	466	0.00085150	6
Labirinto 3	BFS	TRUE	266	266	540	539	0.00098190	7
Labirinto 3	DFS	TRUE	266	266	761	760	0.00169850	10
Labirinto 3	UCS	TRUE	266	266	540	539	0.00085450	7
Labirinto 3	Busca Gulosa	TRUE	266	266	328	327	0.00055160	10
Labirinto 3	A*	TRUE	266	266	469	468	0.00080290	6

4.6 Questões de Análise

1. BFS encontrou o menor caminho? Em quais condições isso ocorre?

Sim. Nos três labirintos testados, a BFS encontrou caminhos com os mesmos custos das buscas ótimas: 68 no Labirinto 01, 314 no Labirinto 02 e 266 no Labirinto 03. A BFS encontra o menor caminho quando todos os movimentos possuem o mesmo custo. Cada movimento ortogonal válido possui custo 1. Por isso, encontrar o caminho com menor profundidade equivale a encontrar o caminho com menor

custo. Dessa forma, a primeira vez que o objetivo é encontrado corresponde ao menor número de passos possível, desde que todos os passos tenham o mesmo custo.

2. DFS encontrou solução rapidamente? A solução foi boa?

A DFS encontrou solução nos três labirintos, mas seu desempenho variou em relação ao esforço de busca. No Labirinto 01, a DFS explorou 169 nós, enquanto BFS e UCS exploraram 85. No Labirinto 03, a DFS explorou 761 nós, enquanto BFS e UCS exploraram 540. Isso mostra que, nesses casos, a DFS exigiu mais exploração. Em termos de qualidade da solução, a DFS encontrou caminhos com o mesmo custo dos demais algoritmos nos três mapas. Porém, isso não significa que a DFS seja ótima em geral.

3. UCS diferiu de BFS quando os custos variaram?

Nos experimentos realizados, os custos não variaram. Todos os movimentos válidos possuíam custo unitário. Por isso, a UCS apresentou comportamento muito semelhante à BFS. A BFS prioriza o caminho com menor número de passos, enquanto a UCS prioriza o caminho com menor custo acumulado ($g(n)$). Portanto, neste trabalho BFS e UCS tiveram resultados semelhantes porque todos os custos eram iguais. Em problemas com custos variáveis, a UCS seria mais adequada para garantir menor custo total.

4. A busca gulosa foi eficiente? Foi ótima?

A Busca Gulosa foi eficiente nos três mapas em termos de nós explorados e expandidos. No Labirinto 01, explorou 69 nós, menos que BFS, DFS, UCS e A*. No Labirinto 02, explorou 357 nós, também menos que os demais algoritmos. No Labirinto 03, explorou 328 nós, novamente o menor valor entre os algoritmos testados. Isso mostra que a heurística de Manhattan guiou bem a busca nos mapas utilizados. Porém, a Busca Gulosa não é ótima em geral. Ela utiliza apenas: $f(n) = h(n)$. Ou seja, considera somente a estimativa de distância até o objetivo e ignora o custo já percorrido. Por isso, pode escolher um caminho que parece aproximar o agente de B, mas que leva a becos sem saída ou desvios longos. Nos experimentos realizados, a Gulosa encontrou caminhos com o mesmo custo dos demais algoritmos. Mesmo assim, ela não garante otimalidade.

5. A* equilibrou qualidade da solução e eficiência?

Sim. O A* apresentou bom equilíbrio entre qualidade da solução e esforço computacional. Ele encontrou caminhos com os mesmos custos de BFS e UCS nos três mapas: 68, 314 e 266. A diferença é que o A* utiliza tanto o custo real já percorrido quanto uma estimativa do custo restante: $f(n) = g(n) + h(n)$. O termo ($g(n)$) evita que o algoritmo ignore o custo já acumulado, enquanto ($h(n)$) orienta a busca na direção do objetivo. Isso faz com que o A* seja mais informado que BFS e UCS, mas mais cuidadoso que a Busca Gulosa. Nos mapas testados, o A* não foi sempre o algoritmo que expandiu menos nós, mas manteve a qualidade ótima da solução e reduziu o esforço em relação a algumas buscas cegas.

6. A heurística utilizada é admissível? Justifique.

Sim. A heurística utilizada foi a distância de Manhattan: $h(n) = |x_n - x_B| + |y_n - y_B|$. Essa heurística é admissível porque nunca superestima o custo real até o objetivo. Como o agente só pode se mover nas direções cima, baixo, esquerda e direita, sem movimentos diagonais, qualquer caminho de uma célula $((x_n, y_n))$ até B precisa realizar pelo menos a diferença vertical mais a diferença horizontal entre os dois pontos. Assim, em um mapa sem obstáculos, a distância de Manhattan seria exatamente o menor número de passos. Por isso, a heurística é admissível para este domínio.

4.7 Visualização da Parte II

Foram geradas visualizações textuais para cada algoritmo e cada labirinto.

Símbolo	Significado
#	parede
espaço	célula livre
A	posição inicial
B	objetivo final
.	caminho encontrado
x	nós explorados ou expandidos

As visualizações dos labirintos foram salvas nos caminhos resultados/vis_labirinto_01, para o labirinto 1. A visualização do labirinto 2, foi salva em resultados/vis_labirinto_02, e a visualização do labirinto 3, ficou em resultados/vis_labirinto_03.

```
resultados > vis_labirinto_01 > ≡ aestrela.txt
1 #####
2 A.#xx...#xx....#...# #
3 #.###.#.###.###.#.#.### #
4 #.#...#.....#...#.#...# #
5 #.#.#####.###.###.# #
6 #...# #.#...# #.# #
7 ##### ##### #.#.### #.# #
8 # # # # #...# #.#x#
9 # # ##### # ##### #.#x#
10 # # # # #...#
11 # ##### # ##### ### ###.#
12 # # # # # # # #...#
13 # # # # # # # #####.###
14 # # # # # ...B
15 #####
```

Figura 1. Visualização da busca A* no Labirinto 01. O símbolo “.” representa o caminho encontrado e “x” representa os nós expandidos/visitados pelo algoritmo.

5. Busca Local com Pontos de Coleta

Nesta etapa, o labirinto contém pontos de coleta obrigatórios (C_i). O agente deve sair de A, visitar todos os pontos de coleta e terminar em B.

O problema deixa de ser: A -> B

e passa a ser: A -> C_n(1) -> C_n(2) -> ... -> C_n(k) -> B, onde n representa uma permutação dos pontos de coleta. Essa etapa transforma o problema em uma tarefa de otimização, pois o agente precisa encontrar uma boa ordem de visita dos pontos obrigatórios.

5.1 Representação da Solução

Uma solução candidata é uma permutação dos pontos de coleta. s = [C₃, C₁, C₂]. Essa solução representa a rota: A -> C₃ -> C₁ -> C₂ -> B

No código, os pontos de coleta foram representados por suas coordenadas na matriz.

5.2 Função de Custo

A função de custo mede o custo total da rota completa:

$$C(s) = d(A, C_{\pi(1)}) + \sum_{i=1}^{k-1} d(C_{\pi(i)}, C_{\pi(i+1)}) + d(C_{\pi(k)}, B), \text{ onde } (d(X,Y))$$

É o custo do menor caminho entre dois pontos do labirinto. Neste trabalho, $(d(X,Y))$ foi calculado com A^* . Antes da execução da busca local, foram pré-computadas as distâncias entre todos os pontos relevantes: $A, C_1, C_2, \dots, C_k, B$. Esse pré-processamento torna a avaliação das soluções mais eficiente.

5.3 Vizinhaça Escolhida

A vizinhaça foi definida por troca de dois pontos da ordem de visitaçaõ.

$$[C_1, C_2, C_3, C_4] \rightarrow [C_1, C_4, C_3, C_2]$$

Essa vizinhaça foi escolhida porque preserva a validade da soluçaõ. Após a troca, todos os pontos de coleta continuam aparecendo exatamente uma vez.

5.4 Hill-Climbing

O Hill-Climbing é um algoritmo de busca local que parte de uma soluçaõ inicial e tenta melhorá-la progressivamente. Neste trabalho, a soluçaõ inicial é uma permutaçaõ aleatória dos pontos de coleta. A cada iteraçaõ, o algoritmo gera vizinhos por troca de dois pontos e escolhe o vizinho com menor custo.

Como o problema é de minimizaçaõ, a regra de decisãõ é:

$$s' = \arg \min_{v \in N(s)} C(v)$$

Se o melhor vizinho tiver custo menor que a soluçaõ atual, o algoritmo se move para esse vizinho. Caso contrário, ele para. O Hill-Climbing é simples e rápido, mas pode ficar preso em mínimos locais.

5.5 Simulated Annealing

O Simulated Annealing também trabalha com soluçaõs candidatas, mas pode aceitar temporariamente soluçaõs piores para escapar de mínimos locais. Se um vizinho piora a soluçaõ, ele ainda pode ser aceito com probabilidade:

$P(\text{aceitar}) = e^{-\Delta E/T}$, onde ΔE é o aumento no custo e (T) é a temperatura atual. A agenda de resfriamento utilizada foi: $T(t) = \alpha^t T_0$

Foram utilizados os seguintes parâmetros:

Parâmetro	Valor
Temperatura inicial (T_0)	1000.0
Taxa de resfriamento α	0.995
Temperatura mínima	0.1
Máximo de iterações	50000
Execuções por algoritmo	30

5.6 Metodologia Experimental

Foram utilizados três mapas com pontos de coleta:

Mapa	Dimensão	Quantidade de coletas
coletas_01.txt	15 x 25	3
coletas_02.txt	27 x 39	3
coletas_03.txt	33 x 49	3

Foram coletadas as seguintes métricas:

- melhor custo encontrado;
- pior custo encontrado;
- custo médio;
- tempo médio;
- número médio de iterações;
- taxa de sucesso;
- curva de convergência.

5.7 Resultados Experimentais

MAPA	ALGORITMO	MELHOR CUSTO	PIOR CUSTO	CUSTO MÉDIO	TEMPO MÉDIO(s)	ITERAÇÕES MÉDIAS	TAXA DE SUCESSO
Coletas 1	Hill-Climbing	144.00	144.00	144.00	0.000043	1.6	100%
Coletas 1	Simulated Annealing	144.00	144.00	144.00	0.006139	1838.0	100%
Coletas 2	Hill-Climbing	314.00	314.00	314.00	0.000039	2.2	100%
Coletas 2	Simulated Annealing	314.00	314.00	314.00	0.006051	1838.0	100%
Coletas 3	Hill-Climbing	190.00	190.00	190.00	0.000036	2.1	100%
Coletas 3	Simulated Annealing	190.00	190.00	190.00	0.006395	1838.0	100%

5.8 Análise Crítica da Parte III

Nos três mapas, Hill-Climbing e Simulated Annealing encontraram o mesmo custo em todas as execuções. No mapa coletas_01, ambos encontraram custo 144. No mapa coletas_02, ambos encontraram custo 314. No mapa coletas_03, ambos encontraram custo 190. Esse resultado indica que, nos mapas testados, o espaço de busca era pequeno e pouco discriminativo.

Cada mapa possuía apenas três pontos de coleta, existiam somente: $3! = 6$ permutações possíveis de visitação. Assim, mesmo algoritmos simples conseguiram alcançar rapidamente boas ordens de visitação. Isso explica por que o Hill-Climbing não apresentou dificuldade relevante e por que o Simulated Annealing não encontrou soluções melhores.

Em mapas com seis ou oito pontos de coleta, o espaço de busca cresceria bastante:

$$6! = 720$$

$$8! = 40320$$

Nesse tipo de cenário, o Hill-Climbing poderia ficar preso em mínimos locais. Isso aconteceria caso uma determinada ordem de visitação não apresentasse nenhum vizinho imediato melhor, considerando a troca de dois pontos de coleta, mesmo existindo uma solução global melhor em outra região do espaço de busca.

O Simulated Annealing, por outro lado, possui maior capacidade de escapar desse tipo de situação, pois pode aceitar temporariamente soluções piores. Essa aceitação é controlada pela temperatura: no início, com temperatura mais alta, o algoritmo tende a explorar mais o espaço de busca; conforme a temperatura diminui, o comportamento se torna mais conservador, favorecendo a convergência.

Nos experimentos realizados o Hill-Climbing convergiu em poucas iterações e apresentou tempo de execução muito baixo. Já o Simulated Annealing executou, em média, cerca de 1838 iterações, apresentando maior custo computacional. Mesmo assim, nos mapas testados, ambos os algoritmos encontraram os mesmos custos finais, indicando que o espaço de busca era pequeno e pouco complexo.

5.9 Questões de Análise da Parte III

1. Hill-Climbing ficou preso em mínimo local?

Não houve evidência prática de que o Hill-Climbing tenha ficado preso em mínimos locais ruins. Em todas as execuções, ele encontrou o mesmo custo obtido pelo Simulated Annealing. O resultado foi influenciado pelo tamanho reduzido do problema. Como havia apenas três pontos de coleta por mapa, o espaço de soluções tinha apenas seis permutações possíveis.

2. Simulated Annealing encontrou soluções melhores?

Não. Nos três mapas, o Simulated Annealing encontrou os mesmos custos do Hill-Climbing: 144, 314 e 190. Para os mapas testados, o Simulated Annealing não trouxe ganho prático na qualidade da solução. Como o problema tinha apenas três pontos de coleta por mapa, havia poucas ordens possíveis, reduzindo a chance de observar diferença significativa entre os algoritmos.

3. Como a temperatura inicial influenciou os resultados?

A temperatura inicial utilizada foi: $T_0 = 1000.0$. Uma temperatura inicial alta aumenta a probabilidade de aceitar soluções piores no começo da execução. Isso favorece a exploração do espaço de soluções e reduz a chance de o algoritmo ficar preso cedo em uma solução local. Nos experimentos realizados, a temperatura inicial influenciou o comportamento do Simulated Annealing em termos de exploração, mas não alterou o custo final. Como o espaço de busca era pequeno, o algoritmo convergiu para o mesmo custo encontrado pelo Hill-Climbing.

4. Como a taxa de resfriamento afetou a convergência?

A taxa de resfriamento utilizada foi: $\alpha = 0.995$. Esse valor reduz a temperatura de forma gradual, permitindo que o algoritmo explore por várias iterações antes de se tornar mais seletivo. Por isso, o Simulated Annealing realizou cerca de 1838 iterações em média. Nos mapas testados, o resfriamento gradual aumentou o número de iterações, mas não melhorou o custo final em relação ao Hill-Climbing.

5. A busca local encontrou sempre a solução ótima?

Os algoritmos encontraram sempre o mesmo custo em todas as execuções. Nos mapas testados, as soluções encontradas provavelmente correspondem às melhores rotas disponíveis. No entanto, não se pode afirmar que a busca local sempre encontra a solução ótima.

6. Qual foi o compromisso entre tempo e qualidade?

Neste experimento, o Hill-Climbing apresentou o melhor compromisso entre tempo e qualidade. Ele encontrou soluções com o mesmo custo do Simulated Annealing, mas com tempo médio muito menor e poucas iterações. O Simulated Annealing teve maior custo computacional, pois realizou cerca de 1838 iterações por execução. Portanto, para os mapas usados, o Hill-Climbing foi mais vantajoso.

5.10 Gráfico de Convergência da Parte III

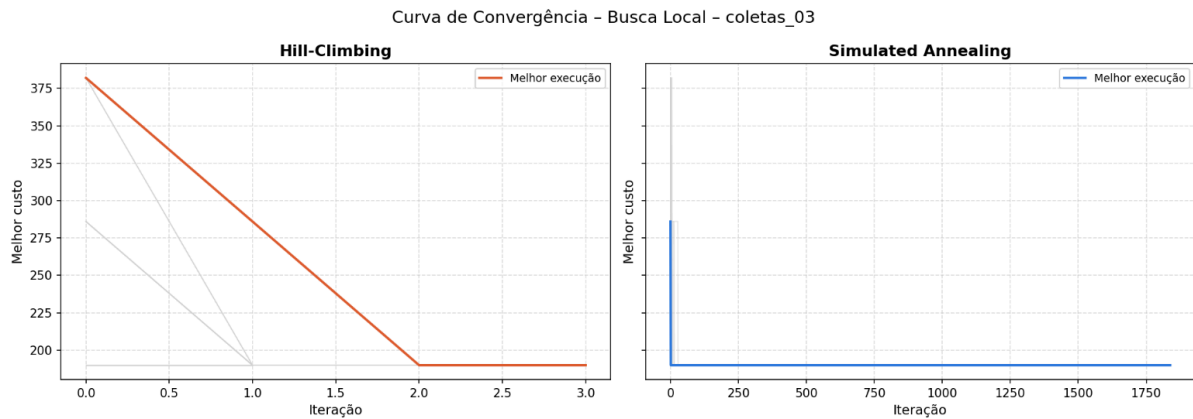


Figura 2. Curva de convergência dos algoritmos Hill-Climbing e Simulated Annealing no mapa coletas_03.

O gráfico de convergência mostra a evolução do melhor custo ao longo das iterações. Nos experimentos realizados, a curva tende a estabilizar rapidamente, pois o espaço de busca possui apenas três pontos de coleta e, portanto, apenas seis permutações possíveis. O Hill-Climbing converge em poucas iterações, pois avalia a vizinhança por troca de pontos e para quando não encontra melhoria. O Simulated Annealing executa mais iterações por causa da agenda de temperatura, mas também converge para o mesmo custo final.

5.11 Visualizações da Parte III

Foram geradas visualizações textuais das rotas otimizadas.

Símbolo	Significado
#	parede
espaço	célula livre
A	posição inicial
B	objetivo final
*	ponto de coleta visitado
.	caminho da rota

Também foram gerados os seguintes arquivos em cada pasta de resultados:

Arquivo	Função
resultados_busca_local.csv	métricas agregadas
resultados_busca_local_raw.csv	custos por execução
convergencia.png	gráfico iteração × melhor custo
boxplot_custos.png	distribuição dos custos
comparativo_custos.png	comparação entre melhor, médio e pior custo
taxa_sucesso.png	taxa de sucesso dos algoritmos

6. Busca Online no Labirinto Desconhecido

Nesta etapa, o agente atua em um labirinto inicialmente desconhecido.

Diferentemente da busca clássica, em que o agente conhece todo o mapa antes de planejar, na busca online o agente precisa explorar o ambiente enquanto se movimenta. O simulador possui acesso ao mapa real, que contém todas as paredes, células livres, posição inicial e objetivo final. Porém, o agente não pode utilizar diretamente esse mapa completo. Ele começa com um mapa interno preenchido com células desconhecidas, representadas por “?”, e atualiza esse modelo conforme percebe novas células.

6.1 Mapa Real e Mapa Interno

Mapa	Função
Mapa real	contém todas as informações do labirinto e é usado apenas pelo simulador
Mapa interno	representação parcial construída pelo agente durante a exploração

O mapa interno começa desconhecido. Apenas a posição inicial A é conhecida no começo. Conforme o agente se movimenta, ele percebe as células vizinhas e substitui os símbolos “?” por informações reais. Essa separação impede que o agente use conhecimento indevido sobre o ambiente. O planejamento é feito apenas sobre o mapa interno, enquanto o mapa real é usado para verificar o resultado das ações e gerar as percepções.

6.2 Estratégia Implementada

A estratégia escolhida foi a Opção A - Replanning com A*.

Nessa estratégia, o agente mantém um mapa interno e, a cada nova percepção, executa novamente o algoritmo A* sobre esse mapa parcialmente conhecido. O agente não executa o caminho completo planejado de uma vez. Em vez disso, executa apenas o próximo movimento, percebe novamente o ambiente, atualiza o mapa interno e replaneja. O ciclo seguido pelo agente é: perceber -> atualizar mapa interno -> planejar -> agir

6.3 Percepção Local

No modo online, o agente percebe apenas uma vizinhança local ao redor da sua posição atual. Considerando raio de percepção ($r=1$), se o agente está na posição $((x,y))$, ele percebe as células ortogonais: $(x-1,y)$, $(x+1,y)$, $(x,y-1)$, $(x,y+1)$.

As células fora do raio de percepção permanecem desconhecidas no mapa interno. Essa limitação é a principal diferença em relação à busca clássica. Na busca clássica, o agente conhece todo o mapa antes de planejar. Na busca online, ele precisa tomar decisões com informação incompleta.

6.4 Funcionamento do Replanning com A*

O algoritmo implementado segue os seguintes passos:

1. inicializar o mapa interno com células desconhecidas ?;
2. posicionar o agente na célula inicial A;
3. perceber as células vizinhas;
4. atualizar o mapa interno com as informações percebidas;
5. executar A* no mapa interno;
6. escolher apenas o próximo passo do caminho planejado;
7. mover o agente no mapa real;
8. repetir o processo até alcançar B ou não haver mais caminho possível.

Durante a execução, o agente pode descobrir paredes que não estavam no seu mapa interno. Quando isso acontece, o plano anterior pode deixar de ser válido, exigindo um novo planejamento.

6.5 Métricas Coletadas

Métrica	Significado
Sucesso	indica se o agente alcançou o objetivo B
Custo Offline	custo ótimo calculado com o mapa completo.
Custo Online	número total de movimentos executados pelo agente online
Razão online/offline	comparação entre custo online e custo ótimo offline
Celulas Reveladas	quantidade de células que deixaram de ser desconhecidas
Celulas Revisitadas	quantidade de vezes que o agente retornou a células já visitadas
Replanejamentos	número de vezes que o agente recalculou a rota

Nós Expandidos	esforço total de planejamento durante as chamadas ao A*
Tempo	tempo total de execução

A métrica é: razão online/offline = custo percorrido pelo agente online / custo ótimo no mapa completo

Quando essa razão é igual a 1, o agente online encontrou um caminho tão bom quanto o agente offline. Quando é maior que 1, o agente paga um custo por não conhecer o mapa completo.

6.6 Resultados Experimentais da Parte IV

MAPA	SUCESSO	CUSTO ONLINE	CUSTO OFFLINE	RAZÃO ONLINE/OFFLINE	CÉLULAS REVELADAS	CÉLULAS REVISITADAS	REPLANEJAMENTOS	NÓS EXPANDIDOS	TEMPOS (s)
Labirinto 01	TRUE	258	68	3.7941	365	95	258	2046	1,029381
Labirinto 02	TRUE	608	314	1.9363	1015	147	608	1905	2,828005
Labirinto 03	TRUE	1172	266	4.4060	1570	453	1172	13867	6,739974

Em todos os mapas, o agente online conseguiu alcançar o objetivo final B. Portanto, a estratégia de Replanning com A* foi capaz de resolver os três labirintos desconhecidos. No entanto, o custo online foi maior do que o custo ótimo offline em todos os casos. Isso era esperado, pois o agente online não conhecia o mapa completo e precisou explorar regiões, visitar células e replanejar várias vezes antes de alcançar o objetivo.

6.7 Análise dos Resultados da Parte IV

A busca online apresentou sucesso nos três mapas, mas o custo percorrido foi maior que o custo ótimo offline em todos os casos. Isso mostra que o agente conseguiu resolver o problema, mas teve um custo adicional por não conhecer o ambiente completo.

No labirinto_01, o custo online foi 258, enquanto o custo ótimo offline foi 68. A razão online/offline foi: $258 / 68 = 3.7941$. Isso indica que o agente percorreu quase quatro vezes o custo do caminho ótimo. Esse resultado sugere que o agente realizou desvios e revisitas provocados pelo desconhecimento inicial do mapa. No labirinto_02, o custo online foi 608 e o custo offline foi 314. A razão online/offline foi: $608 / 314 = 1.9363$. Esse foi o melhor desempenho relativo entre os três mapas. Embora o agente ainda tenha percorrido quase o dobro do caminho ótimo, a diferença foi menor. No labirinto_03, o custo online foi 1172, enquanto o custo offline foi 266. A razão online/offline foi: $1172/266 = 4.4060$. Esse foi o pior resultado relativo. O mapa também apresentou 453 células revisitadas e 13867 nós

expandidos durante o planejamento. Isso indica que o agente encontrou mais regiões que exigiram retorno, replanejamento e exploração adicional.

Como Replanning com A* utiliza apenas o mapa interno disponível no momento, o agente pode planejar caminhos que parecem bons, mas que depois se revelam ruins quando novas paredes são percebidas. Outro ponto importante é que o número de replanejamentos foi igual ao número de movimentos. Isso ocorreu porque o agente replanejou a cada passo.

6.8 Questões de Análise da Parte IV

1. O agente online tomou decisões subótimas? Por quê?

Sim. O agente online tomou decisões subótimas nos três labirintos, pois o custo percorrido foi maior que o custo ótimo offline. No labirinto_01, a razão online/offline foi 3.7941. No labirinto_02, foi 1.9363. No labirinto_03, foi 4.4060. Essas razões mostram que o agente online percorreu caminhos mais longos do que o agente offline. Isso ocorreu porque o agente não possuía acesso ao mapa completo, planejava com base apenas no mapa interno parcial e, ao descobrir novas paredes, precisava corrigir sua rota.

2. Quais informações faltavam ao agente?

Faltavam informações sobre as células ainda não percebidas. No início da execução, o agente não sabia onde estavam todas as paredes, corredores, becos sem saída e rotas alternativas. Como a percepção foi local, com raio ($r=1$), o agente conhecia apenas sua posição atual e as células ortogonais vizinhas. Todo o restante permanecia desconhecido no mapa interno.

3. O mapa interno convergiu para o mapa real?

O mapa interno convergiu apenas parcialmente para o mapa real. O agente revelou as células pelas quais passou e as células próximas à sua trajetória, mas não revelou o labirinto inteiro. Isso pode ser observado nos arquivos `mapa_interno_final.txt`, nos quais algumas células permanecem marcadas como "?". Essas células não foram percebidas porque o agente não precisou passar perto delas para alcançar o objetivo. O objetivo do agente era alcançar B, não mapear todo o ambiente.

4. O agente revisitou muitas células?

Sim. O agente revisitou células nos três mapas:

Mapa	Células Revisitadas
Labirinto 1	95
Labirinto 2	147
Labirinto 3	453

O labirinto_03 apresentou o maior número de revisitas. Isso está relacionado ao maior custo online e à maior razão online/offline. Quanto mais o agente encontra caminhos bloqueados ou regiões pouco úteis, maior a tendência de retornar a células já conhecidas para buscar alternativas. As revisitas também indicam perda de eficiência.

5. Como melhorar a exploração?

Uma primeira melhoria seria priorizar células de fronteira, quando o objetivo não pudesse ser alcançado diretamente no mapa interno, o agente escolheria explorar regiões com maior chance de revelar novas informações úteis. Também seria possível memorizar becos sem saída. Quando o agente descobre que uma região não leva a caminhos úteis, essa informação poderia ser usada para evitar replanejamentos que retornem ao mesmo local. Outra possibilidade seria uma estratégia híbrida poderia combinar Replanning com A* e Online DFS. O A* seria usado quando houvesse um caminho promissor até o objetivo, enquanto o Online DFS auxiliaria em situações de exploração sistemática.

6. O que diferencia a busca online de busca clássica?

Na busca clássica, o agente conhece todo o mapa antes de agir. Assim, ele pode calcular uma solução completa de A até B e depois executar o plano. Na busca online, o agente não conhece o mapa completo. Ele precisa agir para descobrir o ambiente. Por isso, planejamento e execução acontecem de forma intercalada: perceber -> atualizar mapa interno -> planejar -> agir

Essa diferença torna a busca online mais custosa. O agente pode tomar decisões que parecem boas com o conhecimento atual, mas que se tornam ruins quando novas informações são descobertas.

6.9 Visualizações da Parte IV

Foram gerados os seguintes arquivos para cada labirinto:

Arquivo	Função
metricas_online.csv	métricas finais da execução online
trajetoria_online.csv	trajetoria executada passo a passo
mapa_real_com_trajeto.txt	mapa real com o caminho percorrido pelo agente
mapa_interno_final.txt	mapa interno construído pelo agente
mapa_interno_final_com_trajeto.txt	mapa interno final com o trajeto
caminho_otimo_offline.txt	caminho ótimo calculado com o mapa completo
frames/	evolução passo a passo da trajetória online

As pastas geradas foram:

Labirinto 1 resultados/online_labirinto_01

Labirinto 2 resultados/online_labirinto_02

Labirinto 3 resultados/online_labirinto_03

Na visualização:

Símbolo	Significado
#	parede
espaço	célula livre
A	posição inicial
B	objetivo final

.	trajetória percorrida
?	célula ainda desconhecida no mapa interno

7. Visualização Obrigatória

O projeto gerou visualizações para atender às exigências do enunciado.

Exigência de visualização	Como foi atendida
Mapa original	arquivos de mapa .txt e visualizações da Parte II
Caminho encontrado	marcação . nas saídas dos algoritmos
Nos expandidos ou visitados	marcação x na busca clássica
Trajeto online passo a passo	pasta frames/ e arquivo trajetoria_online.csv
Rota otimizada da busca local	visualizações com . e * na Parte III

No corpo do relatório, foram inseridas três figuras principais:

1. uma visualização da Parte II com caminho e nós explorados;
2. o gráfico convergencia.png da Parte III;
3. um mapa interno final da Parte IV.

8. Uso de IA Generativa

Durante o desenvolvimento do trabalho, foi utilizada IA generativa como ferramenta de apoio para compreender o escopo, revisar conceitos, organizar o relatório, interpretar erros de execução, auxiliar na depuração dos códigos e sugerir formas de análise dos resultados. A IA foi utilizada como apoio, mas os resultados foram validados manualmente por meio da execução dos códigos, conferência dos arquivos .csv, inspeção das visualizações e comparação com o comportamento

teórico esperado dos algoritmos. Além desta seção no relatório, foi criado o arquivo separado uso_ia.md, conforme exigido no enunciado.

9. Limitações Gerais

As principais limitações dos experimentos foram:

- os labirintos possuem estrutura bastante restritiva;
- na Parte II, todos os algoritmos encontraram caminhos de mesmo custo;
- na Parte III, cada mapa possui apenas três pontos de coleta;
- com três pontos de coleta, existem apenas ($3! = 6$) permutações possíveis;
- a diferença entre Hill-Climbing e Simulated Annealing ficou mais evidente em tempo e iterações do que em qualidade da solução;
- na Parte IV, o agente online replanejou a cada movimento, o que aumentou o custo computacional;
- o mapa interno da busca online não convergiu completamente para o mapa real, pois o agente não precisou explorar todo o ambiente.

10. Conclusão Geral

Este trabalho apresentou a modelagem e implementação de um agente inteligente capaz de atuar em labirintos discretos sob três abordagens diferentes: busca clássica, busca local e busca online. Na Parte I, o agente foi modelado por meio da estrutura PEAS e classificado como um agente baseado em objetivos com modelo interno. Na Parte II, foram implementados BFS, DFS, UCS, Busca Gulosa e A*. Todos os algoritmos encontraram solução nos três labirintos conhecidos. Como os movimentos tinham custo unitário, os custos encontrados foram iguais em cada mapa, e as diferenças principais apareceram no esforço de busca.

Na Parte III, o problema foi reformulado como uma tarefa de otimização da ordem de visita dos pontos de coleta. Foram implementados Hill-Climbing e Simulated Annealing. Ambos encontraram soluções válidas nos três mapas testados, com custos iguais, mas o Hill-Climbing foi mais eficiente em tempo e número de iterações. Na Parte IV, foi implementada a busca online por Replanning com A*. O agente começou com um mapa interno desconhecido, percebeu células locais, atualizou seu modelo interno e replanejou a cada passo.

Dessa forma, o trabalho demonstrou como diferentes estratégias de busca podem ser aplicadas ao mesmo domínio, destacando as vantagens, limitações e diferenças entre planejamento com mapa conhecido, otimização local e exploração online.